II Multilayer Perceptron Models

A Crash Course to Deep Learning

5

Artificial neural networks form a fascinating area of study, although they can be intimidating when you are just getting started. There are a lot of specialized terms in use to describe the data structures and algorithms used in the field. In this chapter, you will get a crash course in the terminology and processes used in the field of multilayer perceptron artificial neural networks. After completing this chapter, you will know:

- ▷ The building blocks of neural networks, including neurons, weights, and activation functions
- ▷ How the building blocks are used in layers to create networks
- ▷ How networks are trained from example data

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ From Multilayer Perceptrons to Deep Learning
- ▷ Neurons: Weights and Activations
- ▷ Networks of Neurons
- ▷ Training Networks

5.1 From Multilayer Perceptrons to Deep Learning

The field of artificial neural networks is often just called *neural networks* or *multilayer perceptrons*, as multilayer perceptron is perhaps the most useful type of neural network. A perceptron is a single-neuron model that was a precursor to larger neural networks. It is a field of study that investigates how simple models of biological brains can be used to solve difficult computational tasks like the predictive modeling tasks we see in machine learning. The goal is not to create realistic models of the brain but instead to develop robust algorithms and data structures that we can use to model difficult problems.

The power of neural networks comes from their ability to learn the correlations in your training data and how best to relate it to the output variable you want to predict. In this sense, neural networks learn mapping. Mathematically, they are capable of learning any mapping function and have been proven to be a *universal approximation algorithm*. The predictive capability of neural networks comes from the hierarchical or multilayered structure of the networks. The data structure can pick out (learn to represent) features at different scales or resolutions and combine them into higher-order features, for example, from lines to collections of lines to shapes.

Neural networks are usually constructed by stacking up layers. Traditional neural network models has only three layers: The input, output, and one *hidden* layer in-between. Only recently when computers are powerful enough, we learned how to effectively create a neural network with more layers. Hence the term *deep learning* is coined to emphasize the distinction from the older simple neural network models.

5.2 Neurons: Weights and Activations

The building blocks for neural networks are artificial neurons. These are simple computational units that have weighted input signals and produce an output signal using an activation function.

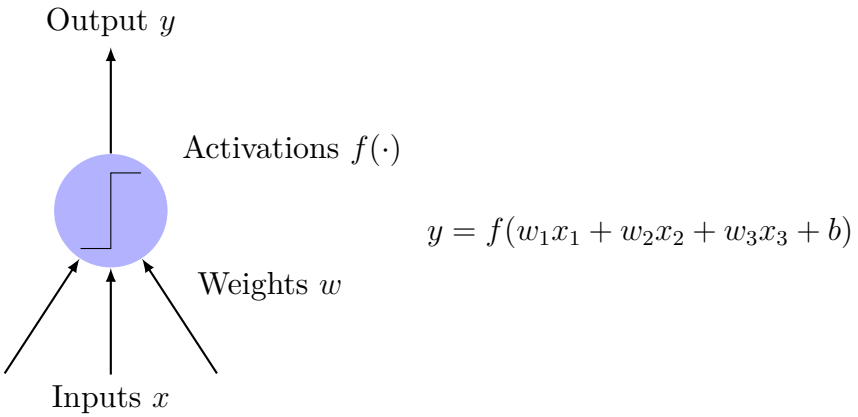


Figure 5.1: Model of a simple neuron with its mathematical representation

Neuron Weights

You may be familiar with linear regression, where the weights on the inputs are very much like the coefficients used in a regression equation. Like linear regression, each neuron also has a bias which can be thought of as an input that always has the value 1.0, and it, too, must be weighted. For example, a neuron may have two inputs, which requires three weights — one for each input and one for the bias.

Weights are often initialized to small random values, such as values from 0 to 0.3, although more complex initialization schemes can be used. Like linear regression, larger weights indicate increased complexity and fragility of the model. Keeping weights in the network small is desirable, and regularization techniques can be used.

Activation

The weighted inputs are summed and passed through an activation function, sometimes called a transfer function. An activation function is a simple mapping of summed weighted input to the output of the neuron. It is called an activation function because it governs the threshold at which the neuron is activated and the strength of the output signal. Historically, simple step activation functions were used. When the summed input was above a threshold of 0.5, for example, then the neuron would output a value of 1.0; otherwise it would output a 0.0.

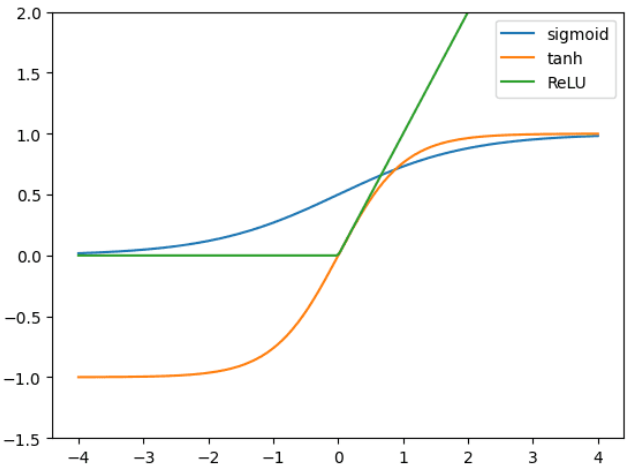


Figure 5.2: Plot of different activation functions

Traditionally, nonlinear activation functions are used. This allows the network to combine the inputs in more complex ways and, in turn, provide a richer capability in the functions they can model. Nonlinear functions like the logistic function, also called the sigmoid function, were used to output a value between 0 and 1 with an S-shaped distribution. The hyperbolic tangent function, also called tanh, were used to output the same distribution over the range −1 to +1. More recently, the rectifier activation function (ReLU) has been shown to provide better results.

5.3 Networks of Neurons

Neurons are arranged into networks of neurons, the figure below is an example. A row of neurons is called a layer, and one network can have multiple layers. The architecture of the neurons in the network is often called the network topology.

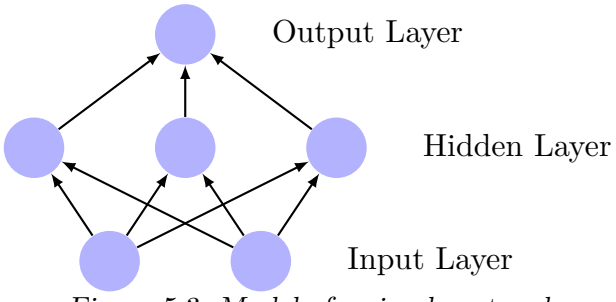


Figure 5.3: Model of a simple network

Input or Visible Layers

The bottom layer that takes input from your dataset is called the visible layer because it is the exposed part of the network. Often a neural network is drawn with a visible layer with one neuron per input value or column in your dataset. These are not actual neurons as described above, but simply drawn as nodes to represent input values that will be passed to the next layer.

Hidden Layers

Layers after the input layer are called hidden layers because they are not directly exposed to the input. The simplest network structure is to have a single neuron in the hidden layer that directly outputs the value. Given increases in computing power and efficient libraries, very deep neural networks can be constructed. Deep learning can refer to having many hidden layers in your neural network. They are deep because they would have been unimaginably slow to train historically but may take seconds or minutes to train using modern techniques and hardware.

Output Layer

The final hidden layer is called the output layer, and it is responsible for outputting a value or vector of values that correspond to the format required for the problem. The choice of activation function in the output layer is strongly constrained by the type of problem that you are modeling. For example:

- ▷ A regression problem may have a single output neuron, and the neuron may have no activation function.
- ▷ A binary classification problem may have a single output neuron and use a sigmoid activation function to output a value between 0 and 1 to represent the probability of the primary class as predicted. This can be turned into a crisp class value by using a threshold of 0.5 and snap values less than the threshold to 0, otherwise to 1.
- ▷ A multiclass classification problem may have multiple neurons in the output layer, one for each class (e.g., three neurons for the three classes in the famous iris flowers classification problem). In this case, a softmax activation function may be used to output a probability of the network predicting each of the class values. Selecting the output with the highest probability can be used to produce a crisp classification value.

5.4 Training Networks

Once configured, the neural network needs to be trained on your dataset.

Data Preparation

You must first prepare your data for training on a neural network. Data must be numerical, for example, real values. If you have categorical data, such as a *sex* attribute with the values "male" and "female", you can convert it to a real-valued representation called one-hot encoding.

This is where one new column is added for each class value (two columns in the case of sex of male and female), and a 0 or 1 is added for each row depending on the class value for that row.

This same one-hot encoding can be used on the output variable in classification problems with more than one class. A binary vector is created from a single column that would be easy to directly compare to the output of the neuron in the network's output layer. That, as described above, would output one value for each class. Neural networks require the input to be scaled consistently. You can rescale it to the range between 0 and 1, this is called normalization. Another popular technique is to standardize it so that the distribution of each column has a mean of zero and a standard deviation of 1. Scaling also applies to image pixel data. Data such as words can be converted to integers, such as the frequency rank of the word in the dataset and other encoding techniques.

Stochastic Gradient Descent

The classical and still preferred training algorithm for neural networks is called stochastic gradient descent. This is where one row of data is exposed to the network at a time as input. The network processes the input upward, activating neurons as it goes to finally produce an output value. This is called a forward pass on the network. It is the type of pass that is also used after the network is trained in order to make predictions on new data.

The output of the network is compared to the expected output, and an error is calculated. This error is then propagated back through the network, one layer at a time, and the weights are updated according to the amount they contributed to the error. This clever bit of math is called the backpropagation algorithm. The process is repeated for all of the examples in your training data. One round of updating the network for the entire training dataset is called an epoch. A network may be trained for tens, hundreds, or many thousands of epochs.

Weight Updates

The weights in the network can be updated from the errors calculated for each training example, and this is called *online learning*. It can result in fast but also chaotic changes to the network. Alternatively, the errors can be saved across all the training examples, and the network can be updated at the end. This is called *batch learning* and is often more stable.

Because datasets are so large and because of computational efficiencies, the size of the batch, the number of examples the network is shown before an update, is often reduced to a small number, such as tens or hundreds of examples. The amount that weights are updated is controlled by a configuration parameter called the *learning rate*. It is also called the *step size* and controls the step or change made to network weights for a given error. Often small learning rates are used, such as 0.1 or 0.01 or smaller. The update equation can be complemented with additional configuration terms that you can set.

- ▷ **Momentum** is a term that carry over the weight update of the previous step to the current, so the weight update will continue in the same direction even when there is less error being calculated.

- ▷ **Learning Rate Decay** is used to decrease the learning rate over epochs to allow the network to make large changes to the weights at the beginning and smaller fine-tuning changes later in the training schedule.

Prediction

Once a neural network has been trained, it can be used to make predictions. You can make predictions on test or validation data in order to estimate the skill of the model on unseen data. You can also deploy it operationally and use it to make predictions continuously. The network topology and the final set of weights are all you need to save from the model. Predictions are made by providing the input to the network and performing a forward-pass, allowing it to generate an output you can use as a prediction.

5.5 More Resources

There are decades of papers and books on the topic of artificial neural networks. If you are new to the field, check out the following resources as further reading:

Books

Christopher M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1996.
https://www.amazon.com/dp/0198538642

Russell D. Reed and Robert J. Marks II. *Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks*. MIT Press, 1999.
https://www.amazon.com/dp/0262527014

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
https://www.amazon.com/dp/0262035618
(Online version at http://www.deeplearningbook.org).

5.6 Summary

In this chapter, you discovered artificial neural networks for machine learning. You learned:

- ▷ How neural networks are not models of the brain but are instead computational models for solving complex machine learning problems.
- ▷ That neural networks are comprised of neurons that have weights and activation functions.
- ▷ The networks are organized into layers of neurons and are trained using stochastic gradient descent.
- ▷ That it is a good idea to prepare your data before training a neural network model.

In the next chapter, you will learn what PyTorch provides to build a deep learning model.

Multilayer Perceptron Building Blocks in PyTorch

6

The PyTorch library is for deep learning. Deep learning, indeed, is just another name for a large-scale neural network or multilayer perceptron network. In its simplest form, multilayer perceptrons are a sequence of layers connected in tandem. In this chapter, you will discover the simple components you can use to create neural networks and simple deep learning models in PyTorch.

Let's get started.

Overview

This chapter is in six parts; they are:

- ▷ Neural Network Models in PyTorch
- ▷ Model Inputs
- ▷ Layers, Activations, and Layer Properties
- ▷ Loss Functions and Model Optimizers
- ▷ Model Training and Inference
- ▷ Examination of a Model

6.1 Neural Network Models in PyTorch

PyTorch can do a lot of things, but the most common use case is to build a deep learning model. The simplest model can be defined using `Sequential` class, which is just a linear stack of layers connected in tandem. You can create a `Sequential` model and define all the layers in one shot; for example:

```
import torch
import torch.nn as nn

model = nn.Sequential(...)
```

Listing 6.1: Building a MLP with the `Sequential` class

You should have all your layers defined inside the parentheses in the processing order from input to output. For example:

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(764, 100),
    nn.ReLU(),
    nn.Linear(100, 50),
    nn.ReLU(),
    nn.Linear(50, 10),
    nn.Sigmoid()
)
print(model)
```

Listing 6.2: An example MLP model

The other way of using `Sequential` is to pass in an ordered dictionary in which you can assign names to each layer:

```
from collections import OrderedDict
import torch.nn as nn

model = nn.Sequential(OrderedDict([
    ('dense1', nn.Linear(764, 100)),
    ('act1', nn.ReLU()),
    ('dense2', nn.Linear(100, 50)),
    ('act2', nn.ReLU()),
    ('output', nn.Linear(50, 10)),
    ('outact', nn.Sigmoid()),
]))
print(model)
```

Listing 6.3: An MLP model built using `OrderedDict`

And if you would like to build the layers one by one instead of doing everything in one shot, you can do the following:

```
import torch.nn as nn

model = nn.Sequential()
model.add_module("dense1", nn.Linear(8, 12))
model.add_module("act1", nn.ReLU())
model.add_module("dense2", nn.Linear(12, 8))
model.add_module("act2", nn.ReLU())
model.add_module("output", nn.Linear(8, 1))
model.add_module("outact", nn.Sigmoid())
print(model)
```

Listing 6.4: An MLP model created by adding one layer at a time

You will find this helpful in a more complex case where you need to build a model based on some conditions.

6.2 Model Inputs

The first layer in your model hints at the shape of the input. In the example above, you have `nn.Linear(764, 100)` as the first layer. Depending on the different layer type you use, the arguments may bear different meanings. But in this case, it is a `Linear` layer (also known as a dense layer or fully connected layer), and the two arguments tell the input and output dimensions of *this layer*.

Note that the size of a batch is implicit. In this example, you should pass in a PyTorch tensor of shape `(n, 764)` into this layer and expect a tensor of shape `(n, 100)` in return, where `n` is the size of a batch.

6.3 Layers, Activations, and Layer Properties

There are many kinds of neural network layers defined in PyTorch. In fact, it is easy to define your own layer if you want to. Below are some common layers that you may see often:

- ▷ `nn.Linear(input, output)`: The fully-connected layer
- ▷ `nn.Conv2d(in_channel, out_channel, kernel_size)`: The 2D convolution layer, popular in image processing networks
- ▷ `nn.Dropout(probability)`: Dropout layer, usually added to a network to introduce regularization
- ▷ `nn.Flatten()`: Reshape a high-dimensional input tensor into 1-dimensional (per each sample in a batch)

Besides layers, there are also activation functions. These are functions applied to each element of a tensor. Usually, you take the output of a layer and apply the activation before feeding it as input to a subsequent layer. Some common activation functions are:

- ▷ `nn.ReLU()`: Rectified linear unit, the most common activation nowadays
- ▷ `nn.Sigmoid()` and `nn.Tanh()`: Sigmoid and hyperbolic tangent functions, which are the usual choice in older literature
- ▷ `nn.Softmax()`: To convert a vector into probability-like values; popular in classification networks

You can find a list of all the different layers and activation functions in PyTorch's documentation.

The design of PyTorch is very modular. Therefore, you don't have much to adjust in each component. Take this `Linear` layer as an example. You can only specify the input and output shape but not other details, such as how to initialize the weights. However, almost all the components can take two additional arguments: the device and the data type.

A PyTorch device specifies where this layer will execute. Normally, you choose between the CPU and the GPU or omit it and let PyTorch decide. To specify a device, you do the following:

```
nn.Linear(764, 100, device="cpu")
```

Listing 6.5: Creating part of a model at CPU

or (CUDA means a supported nVidia GPU):

```
nn.Linear(764, 100, device="cuda:0")
```

Listing 6.6: Creating part of a model at GPU

The data type argument (`dtype`) specifies what kind of data type this layer should operate on. Usually, it is a 32-bit float, and usually, you don't want to change that. But if you need to specify a different type, you must do so using PyTorch types, e.g.,

```
nn.Linear(764, 100, dtype=torch.float16)
```

Listing 6.7: Setting operation data type

6.4 Loss Function and Model Optimizers

A neural network model is a sequence of matrix operations. The matrices that are independent of the input and kept inside the model are called weights. Training a neural network will *optimize* these weights so that they produce the output you want. In deep learning, the algorithm to optimize these weights is gradient descent.

There are many variations of gradient descent. You can make your choice by preparing an optimizer for your model. It is not part of the model, but you will use it alongside the model during training. The way you use it includes defining a *loss function* and minimizing the loss function using the optimizer. The loss function will give a *distance score* to tell how far away the model's output is from your desired output. It compares the output tensor of the model to the expected tensor, which is called the *label* or the *ground truth* in a different context. Because it is provided as part of the training dataset, a neural network model is a supervised learning model.

In PyTorch, you can simply take the model's output tensor and manipulate it to calculate the loss. But you can also make use of the functions provided in PyTorch for that, e.g.,

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(output, label)
```

Listing 6.8: Creating and using a loss function

In this example, the `loss_fn` is a function, and `loss` is a tensor that supports automatic differentiation. You can trigger the differentiation by calling `loss.backward()`.

Below are some common loss functions in PyTorch:

- ▷ `nn.MSELoss()`: Mean square error, useful in regression problems
- ▷ `nn.CrossEntropyLoss()`: Cross-entropy loss, useful in classification problems
- ▷ `nn.BCELoss()`: Binary cross-entropy loss, useful in binary classification problems

Creating an optimizer is similar:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Listing 6.9: Creating an optimizer

All optimizers require a list of all parameters that it needs to optimize. It is because the optimizer is created outside the model, and you need to tell it where to look for the parameters (i.e., model weights). Then, the optimizer will take the gradient as computed by the `backward()` function call and apply it to the parameters based on the optimization algorithm.

This is a list of some common optimizers:

- ▷ `torch.optim.Adam()`: The Adam algorithm (adaptive moment estimation)
- ▷ `torch.optim.NAdam()`: The Adam algorithm with Nesterov momentum
- ▷ `torch.optim.SGD()`: Stochastic gradient descent
- ▷ `torch.optim.RMSprop()`: The RMSprop algorithm

You can find a list of all provided loss functions and optimizers in PyTorch's documentation, where you can also learn about the mathematical formula of each optimization algorithm.

6.5 Model Training and Inference

PyTorch doesn't have a dedicated function for model training and evaluation. A defined model by itself is like a function. You pass in an input tensor and get back the output tensor. Therefore, it is your responsibility to write the training loop. A minimal training loop is like the following:

```
for n in range(num_epochs):
    y_pred = model(X)
    loss = loss_fn(y_pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Listing 6.10: Training a model with an optimizer

If you already have a model, you can simply take `y_pred = model(X)` and use the output tensor `y_pred` for other purposes. That's how you use the model for prediction or inference. A model, however, does not expect one input sample but a batch of input samples in one tensor. If the model is to take an input vector (which is one-dimensional), you should provide a two-dimensional tensor to the model. Usually, in the case of inference, you deliberately create a batch of one sample.

6.6 Examination of a Model

Once you have a model, you can check what it is by printing it:

```
print(model)
```

Listing 6.11: Printing a model

This will give you, for example, the following:

```
Sequential(
  (0): Linear(in_features=8, out_features=12, bias=True)
  (1): ReLU()
  (2): Linear(in_features=12, out_features=8, bias=True)
  (3): ReLU()
  (4): Linear(in_features=8, out_features=1, bias=True)
  (5): Sigmoid()
)
```

Output 6.1: A model printed

If you would like to save the model, you can use the `pickle` library from Python. But you can also access it using PyTorch:

```
torch.save(model, "my_model.pth")
```

Listing 6.12: Saving a PyTorch model

This way, you have the entire model object saved. You can retrieve the model with:

```
model = torch.load("my_model.pth")
```

Listing 6.13: Reloading a PyTorch model

But the recommended way of saving a model is to leave the model design in code and keep only the weights (more details on Chapter 13). You can do so with:

```
torch.save(model.state_dict(), "my_model.pth")
```

Listing 6.14: Saving a PyTorch model with the `save()` function

The `state_dict()` function extracts only the states (i.e., weights in a model). To retrieve it, you need to rebuild the model from scratch and then load the weights like this:

```
model = nn.Sequential(...)
model.load_state_dict(torch.load("my_model.pth"))
```

Listing 6.15: Reloading a model using PyTorch's `load()` function

6.7 Further Readings

You can learn more about how to create simple neural networks and deep learning models in PyTorch using the following resources:

Articles

PyTorch tutorials.
https://pytorch.org/tutorials/
torch.nn. PyTorch API.
https://pytorch.org/docs/stable/nn.html
torch.optim. PyTorch API.
https://pytorch.org/docs/stable/optim.html

6.8 Summary

In this chapter, you discovered the PyTorch API that you can use to create artificial neural networks and deep learning models. Specifically, you learned about the life cycle of a PyTorch model, including:

- ▷ Constructing a model
- ▷ Creating and adding layers and activations
- ▷ Preparing a model for training and inference

In the next chapter, you will build your first deep learning model using PyTorch.

Your First Neural Network in PyTorch, Step by Step

7

PyTorch is a powerful Python library for building deep learning models. It provides everything you need to define and train a neural network and use it for inference. You don't need to write much code to complete all this. In this chapter, you will discover how to create your first deep learning neural network model in Python using PyTorch. After completing this chapter, you will know:

- ▷ How to load a CSV dataset and prepare it for use with PyTorch
- ▷ How to define a Multilayer Perceptron model in PyTorch
- ▷ How to train and evaluate a PyTorch model on a validation dataset

Let's get started.

Overview

There is not a lot of code required. You will go over it slowly so that you will know how to create your own models in the future. The steps you will learn in this chapter are as follows:

- ▷ Load Data
- ▷ Define PyTorch Model
- ▷ Define Loss Function and Optimizers
- ▷ Run a Training Loop
- ▷ Evaluate the Model
- ▷ Make Predictions

7.1 Load Data

The first step is to define the functions and classes you intend to use in this chapter. You will use the NumPy library to load your dataset and the PyTorch library for deep learning models.

The imports required are listed below:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

Listing 7.1: Dependencies

You can now load your dataset. You can download it here:

▷ https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

In this chapter, you will use the Pima Indians onset of diabetes dataset. This has been a standard machine learning dataset since the early days of the field. It describes patient medical record data for Pima Indians and whether they had an onset of diabetes within five years.

It is a binary classification problem (onset of diabetes as 1 or not as 0). All the input variables that describe each patient are transformed and numerical. This makes it easy to use directly with neural networks that expect numerical input and output values and is an ideal choice for our first neural network in PyTorch.

Download the dataset and place it in your local working directory, the same location as your Python file. Save it with the filename `pima-indians-diabetes.csv`. Take a look inside the file; you should see rows of data like the following:

```
6,148,72,35,0,33.6,0.627,50,1
1,85,66,29,0,26.6,0.351,31,0
8,183,64,0,0,23.3,0.672,32,1
1,89,66,23,94,28.1,0.167,21,0
0,137,40,35,168,43.1,2.288,33,1
...
```

Output 7.1: Dataset in CSV format

You can now load the file as a matrix of numbers using the NumPy function `loadtxt()`. There are eight input variables and one output variable (the last column). You will be learning a model to map rows of input variables (X) to an output variable (y), which is often summarized as $y = f(X)$. The variables are summarized as follows:

Input Variables (X):

1. Number of times pregnant
2. Plasma glucose concentration at 2 hours in an oral glucose tolerance test
3. Diastolic blood pressure (mm Hg)
4. Triceps skin fold thickness (mm)
5. 2-hour serum insulin (μIU/ml)
6. Body mass index (weight in kg/(height in m)^2)
7. Diabetes pedigree function
8. Age (years)

Output Variables (y):

▷ Class label (0 or 1)

Once the CSV file is loaded into memory, you can split the columns of data into input and output variables.

The data will be stored in a 2D array where the first dimension is rows and the second dimension is columns, e.g., (rows, columns). You can split the array into two arrays by selecting subsets of columns using the standard NumPy slice operator ":". You can select the first eight columns from index 0 to index 7 via the slice `0:8`. You can then select the output column (the 9th variable) via index 8.

```
...

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
```

Listing 7.2: Loading data from a CSV file

But these data should be converted to PyTorch tensors first. One reason is that PyTorch usually operates in a 32-bit floating point while NumPy, by default, uses a 64-bit floating point. Mix-and-match is not allowed in most operations. Converting to PyTorch tensors can avoid the implicit conversion that may cause problems. You can also take this chance to correct the shape to fit what PyTorch would expect, e.g., prefer $n \times 1$ matrix over n-vectors.

To convert, create a tensor out of NumPy arrays:

```
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
```

Listing 7.3: Converting a NumPy array into PyTorch tensor

You are now ready to define your neural network model.

7.2 Define the Model

Indeed, there are two ways to define a model in PyTorch. The goal is to make it like a function that takes an input and returns an output.

A model can be defined as a sequence of layers. You create a `Sequential` model with the layers listed out. The first thing you need to do to get this right is to ensure the first layer has the correct number of input features. In this example, you can specify the input dimension `8` for the eight input variables as one vector.

The other parameters for a layer or how many layers you need for a model is not an easy question. You may use heuristics to help you design the model, or you can refer to other people's designs in dealing with a similar problem. Often, the best neural network structure is found through a process of trial-and-error experimentation. Generally, you need a network

large enough to capture the structure of the problem but small enough to make it fast. In this example, let's use a fully-connected network structure with three layers.

Fully connected layers or dense layers are defined using the `Linear` class in PyTorch. It simply means an operation similar to matrix multiplication. You can specify the number of inputs as the first argument and the number of outputs as the second argument. The number of outputs is sometimes called the number of neurons or number of nodes in the layer.

You also need an activation function *after* the layer. If not provided, you just take the output of the matrix multiplication to the next step, or sometimes you call it using linear activation, hence the name of the layer.

In this example, you will use the *rectified linear unit* activation function, referred to as ReLU, on the first two layers and the sigmoid function in the output layer.

A sigmoid on the output layer ensures the output is between 0 and 1, which is easy to map to either a probability of class 1 or snap to a hard classification of either class by a cut-off threshold of 0.5. In the past, you might have used sigmoid and tanh activation functions for all layers, but it turns out that sigmoid activation can lead to the problem of vanishing gradient in deep neural networks, and ReLU activation is found to provide better performance in terms of both speed and accuracy.

You can make a complete model by adding each layer such that:

- ▷ The model expects rows of data with 8 variables (the first argument at the first layer set to `8`)
- ▷ The first hidden layer has 12 neurons, followed by a ReLU activation function
- ▷ The second hidden layer has 8 neurons, followed by another ReLU activation function
- ▷ The output layer has one neuron, followed by a sigmoid activation function

```
...

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
```

Listing 7.4: Creating an MLP model

You can check the model by printing it out as follows:

```
print(model)
```

Listing 7.5: Showing the MLP model created

You will see:

```
Sequential(
  (0): Linear(in_features=8, out_features=12, bias=True)
  (1): ReLU()
  (2): Linear(in_features=12, out_features=8, bias=True)
  (3): ReLU()
  (4): Linear(in_features=8, out_features=1, bias=True)
  (5): Sigmoid()
)
```

Output 7.2: The MLP model created

You are free to change the design and see if you get a better or worse result than the subsequent part of this chapter.

But note that, in PyTorch, there is a more verbose way of creating a model. The model above can be created as a Python `class` inherited from the `nn.Module`:

```
...

class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden1 = nn.Linear(8, 12)
        self.act1 = nn.ReLU()
        self.hidden2 = nn.Linear(12, 8)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(8, 1)
        self.act_output = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.hidden1(x))
        x = self.act2(self.hidden2(x))
        x = self.act_output(self.output(x))
        return x

model = PimaClassifier()
print(model)
```

Listing 7.6: The same MLP model created as a subclass of `nn.Module`

In this case, the model printed will be:

```
PimaClassifier(
  (hidden1): Linear(in_features=8, out_features=12, bias=True)
  (act1): ReLU()
  (hidden2): Linear(in_features=12, out_features=8, bias=True)
  (act2): ReLU()
  (output): Linear(in_features=8, out_features=1, bias=True)
  (act_output): Sigmoid()
)
```

Output 7.3: The MLP model created

In this approach, a class needs to have all the layers defined in the constructor because you need to prepare all its components when it is created, but the input is not yet provided.

Note that you also need to call the parent class's constructor (the line `super().__init__()`) to bootstrap your model. You also need to define a `forward()` function in the class to tell, if an input tensor `x` is provided, how you produce the output tensor in return.

You can see from the output above that the model remembers how you named each layer.

7.3 Preparation for Training

A defined model is ready for training, but you need to specify what the goal of the training is. In this example, the data has the input features X and the output label y. You want the neural network model to produce an output that is as close to y as possible. Training a network means finding the best set of weights to map inputs to outputs in your dataset. The *loss function* is the metric to measure the prediction's distance to y. In this example, you should use binary cross-entropy because it is a binary classification problem.

Once you decide on the loss function, you also need an optimizer. The optimizer is the algorithm you use to adjust the model weights progressively to produce a better output. There are many optimizers to choose from, and in this example, Adam is used. This popular version of gradient descent can automatically tune itself and gives good results in a wide range of problems.

```
loss_fn = nn.BCELoss()   # binary cross-entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 7.7: Creating a loss function and an optimizer

The optimizer usually has some configuration parameters. Most notably, the learning rate `lr`. But all optimizers needs to know what to optimize. Therefore we pass on `model.parameters()`, which is a generator of all parameters from the model we created.

7.4 Training a Model

You have defined your model, the loss metric, and the optimizer. It is ready for training by executing the model on some data.

Training a neural network model usually takes in epochs and batches. They are idioms for how data is passed to a model:

- ▷ *Epoch*: Passes the entire training dataset to the model once
- ▷ *Batch*: One or more samples passed to the model, from which the gradient descent algorithm will be executed for one iteration

Simply speaking, the entire dataset is split into batches, and you pass the batches one by one into a model using a training loop. Once you have exhausted all the batches, you have finished one epoch. Then you can start over again with the same dataset and start the second epoch, continuing to refine the model. This process repeats until you are satisfied with the model's output.

The size of a batch is limited by the system's memory. Also, the number of computations required is linearly proportional to the size of a batch. The total number of batches over many epochs is how many times you run the gradient descent to refine the model. It is a trade off that you want more iterations for the gradient descent so you can produce a better model, but at the same time, you do not want the training to take too long time to complete. The number of epochs and the size of a batch can be chosen experimentally by trial and error.

The goal of training a model is to ensure it learns a good enough mapping of input data to output classification. It will not be perfect, and errors are inevitable. Usually, you will see the amount of error reducing when in the later epochs, but it will eventually level out. This is called model convergence.

The simplest way to build a training loop is to use two nested for-loops, one for epochs and one for batches:

```
n_epochs = 100
batch_size = 10

for epoch in range(n_epochs):
    for i in range(0, len(X), batch_size):
        Xbatch = X[i:i+batch_size]
        y_pred = model(Xbatch)
        ybatch = y[i:i+batch_size]
        loss = loss_fn(y_pred, ybatch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f'Finished epoch {epoch}, latest loss {loss}')
```

Listing 7.8: The training loop

When this runs, it will print the following:

```
Finished epoch 0, latest loss 0.6271069645881653
Finished epoch 1, latest loss 0.6056771874427795
Finished epoch 2, latest loss 0.5916517972946167
Finished epoch 3, latest loss 0.5822567939758301
Finished epoch 4, latest loss 0.5682642459869385
Finished epoch 5, latest loss 0.5640913248062134
...
```

Output 7.4: Progress printed while training the model

7.5 Evaluate the Model

You have trained our neural network on the entire dataset, and you can evaluate the performance of the network on the same dataset. This will only give you an idea of how well you have modeled the dataset (e.g., train accuracy) but no idea of how well the algorithm might perform on new data. This was done for simplicity, but ideally, you could separate your data into train and test datasets for training and evaluation of your model.

You can evaluate your model on your training dataset in the same way you invoked the model in training. This will generate predictions for each input, but then you still need to compute a score for the evaluation. This score can be the same as your loss function or something different. Because you are doing binary classification, you can use accuracy as your evaluation score by converting the output (a floating point in the range of 0 to 1) to an integer (0 or 1) and compare to the label we know.

This is done as follows:

```
# compute accuracy (no_grad is optional)
with torch.no_grad():
    y_pred = model(X)

accuracy = (y_pred.round() == y).float().mean()
print(f"Accuracy {accuracy}")
```

Listing 7.9: Evaluating the trained model

The `round()` function rounds off the floating point to the nearest integer. The `==` operator compares and returns a Boolean tensor, which can be converted to floating point numbers 1.0 and 0.0. The `mean()` function will provide you the count of the number of 1's (i.e., prediction matches the label) divided by the total number of samples. The `no_grad()` context is optional but suggested, so you relieve `y_pred` from remembering how it comes up with the number since you are not going to do differentiation on it.

Putting everything together, the following is the complete code.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# define the model
model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# train the model
loss_fn   = nn.BCELoss()  # binary cross-entropy
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100
batch_size = 10

for epoch in range(n_epochs):
    for i in range(0, len(X), batch_size):
        Xbatch = X[i:i+batch_size]
        y_pred = model(Xbatch)
        ybatch = y[i:i+batch_size]
        loss = loss_fn(y_pred, ybatch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f'Finished epoch {epoch}, latest loss {loss}')

# compute accuracy (no_grad is optional)
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print(f"Accuracy {accuracy}")
```

Listing 7.10: Complete code to build, train, and evaluate an MLP model

You can copy all the code into your Python file and save it as "`pytorch_network.py`" in the same directory as your data file "`pima-indians-diabetes.csv`". You can then run the Python file as a script from your command line.

Running this example, you should see that the training loop progresses on each epoch with the loss with the final accuracy printed last. Ideally, you would like the loss to go to zero and the accuracy to go to 1.0 (e.g., 100%). This is not possible for any but the most trivial machine learning problems. Instead, you will always have some error in your model. The goal is to choose a model configuration and training configuration that achieves the lowest loss and highest accuracy possible for a given dataset.

Neural networks are stochastic algorithms, meaning that the same algorithm on the same data can train a different model with different skill each time the code is run. This is a feature, not a bug. The variance in the performance of the model means that to get a reasonable approximation of how well your model is performing, you may need to fit it many times and calculate the average of the accuracy scores. For example, below are the accuracy scores from re-running the example five times:

```
Accuracy: 0.7604166865348816
Accuracy: 0.7838541865348816
Accuracy: 0.7669270634651184
Accuracy: 0.7721354365348816
Accuracy: 0.7669270634651184
```

Output 7.5: Various accuracy obtained by re-running the program

You can see that all accuracy scores are around 77%, roughly.

7.6 Make Predictions

You can adapt the above example and use it to generate predictions on the training dataset, pretending it is a new dataset you have not seen before. Making predictions is as easy as calling the model as if it is a function. You are using a sigmoid activation function on the output layer so that the predictions will be a probability in the range between 0 and 1. You can easily convert them into a crisp binary prediction for this classification task by rounding them. For example:

```
...

# make probability predictions with the model
predictions = model(X)
# round predictions
rounded = predictions.round()
```

Listing 7.11: Making prediction from the trained model

Alternately, you can convert the probability into 0 or 1 to predict crisp classes directly; for example:

```
...
# make class predictions with the model
predictions = (model(X) > 0.5).int()
```

Listing 7.12: Converting a probability output into the class label

The complete example below makes predictions for each example in the dataset, then prints the input data, predicted class, and expected class for the first five examples in the dataset.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# define the model
class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden1 = nn.Linear(8, 12)
        self.act1 = nn.ReLU()
        self.hidden2 = nn.Linear(12, 8)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(8, 1)
```

```
        self.act_output = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.hidden1(x))
        x = self.act2(self.hidden2(x))
        x = self.act_output(self.output(x))
        return x

model = PimaClassifier()
print(model)

# train the model
loss_fn   = nn.BCELoss()  # binary cross-entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100
batch_size = 10

for epoch in range(n_epochs):
    for i in range(0, len(X), batch_size):
        Xbatch = X[i:i+batch_size]
        y_pred = model(Xbatch)
        ybatch = y[i:i+batch_size]
        loss = loss_fn(y_pred, ybatch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# compute accuracy
y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print(f"Accuracy {accuracy}")

# make class predictions with the model
predictions = (model(X) > 0.5).int()
for i in range(5):
    print('%s => %d (expected %d)' % (X[i].tolist(), predictions[i], y[i]))
```

Listing 7.13: Complete code to use the model for prediction

This code uses a different way of building the model but should functionally be the same as before. After the model is trained, predictions are made for all examples in the dataset, and the input rows and predicted class value for the first five examples are printed and compared to the expected class value. You can see that most rows are correctly predicted. In fact, you can expect about 77% of the rows to be correctly predicted based on your estimated performance of the model in the previous section.

```
[6.0, 148.0, 72.0, 35.0, 0.0, 33.599998474121094, 0.6269999742507935, 50.0] => 1 (expected 1)
[1.0, 85.0, 66.0, 29.0, 0.0, 26.600000381469727, 0.35100001096725464, 31.0] => 0 (expected 0)
[8.0, 183.0, 64.0, 0.0, 0.0, 23.299999237060547, 0.671999990940094, 32.0] => 1 (expected 1)
[1.0, 89.0, 66.0, 23.0, 94.0, 28.100000381469727, 0.16699999570846558, 21.0] => 0 (expected 0)
[0.0, 137.0, 40.0, 35.0, 168.0, 43.099998474121094, 2.2880001068115234, 33.0] => 1 (expected 1)
```

Output 7.6: Inputs to the model and its predictions

7.7 Further Reading

To learn more about deep learning and PyTorch, take a look at some of these:

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
https://www.amazon.com/dp/0262035618
(Online version at http://www.deeplearningbook.org).

PyTorch documentation.
https://pytorch.org/docs/stable/index.html

Build the Neural Network. PyTorch tutorials.
https://pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html

Optimizing Model Parameters. PyTorch tutorials.
https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

7.8 Summary

In this chapter, you discovered how to create your first neural network model using PyTorch. Specifically, you learned the key steps in using PyTorch to create a neural network or deep learning model step by step, including:

- How to load data
- How to define a neural network in PyTorch
- How to train a model on data
- How to evaluate a model
- How to make predictions with the model

In this chapter, a simple workflow is presented but every step can be deep-dived. In the next chapter, you will look into the training loop and learn how to polish it.

Creating a Training Loop for Your Models

8

PyTorch provides a lot of building blocks for a deep learning model, but a training loop is not part of them. It is a flexibility that allows you to do whatever you want during training, but some basic structure is universal across most use cases.

In this chapter, you will see how to make a training loop that provides essential information for your model training, with the option to allow any information to be displayed. After completing this chapter, you will know:

- ▷ The basic building block of a training loop
- ▷ How to use tqdm to display training progress

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Elements of Training a Deep Learning Model
- ▷ Collecting Statistics During Training
- ▷ Using tqdm to Report the Training Progress

8.1 Elements of Training a Deep Learning Model

As with all machine learning models, the model design specifies the algorithm to manipulate an input and produce an output. But in the model, there are parameters that you need to fine-tune to achieve that. These model parameters are also called the weights, biases, kernels, or other names depending on the particular model and layers. Training is to feed in the sample data to the model so that an optimizer can fine-tune these parameters.

When you train a model, you usually start with a dataset. Each dataset is a fairly large number of data samples. When you get a dataset, it is recommended to split it into two portions: the training set and the test set. The training set is further split into batches and used in the training loop to drive the gradient descent algorithms. The test set, however, is used as a benchmark to tell how good your model is. Usually, you do not use the training set

as a metric but take the test set, which is not seen by the gradient descent algorithm, so you can tell if your model fits well to the unseen data.

Overfitting is when the model fits too well to the training set (i.e., at very high accuracy) but performs significantly worse in the test set. Underfitting is when the model cannot even fit well to the training set. Naturally, you don't want to see either on a good model.

Training of a neural network model is in epochs. Usually, one epoch means you run through the entire training set once, although you only feed one batch at a time. It is also customary to do some housekeeping tasks at the end of each epoch, such as benchmarking the partially trained model with the test set, checkpointing the model, deciding if you want to stop the training early, and collecting training statistics, and so on.

In each epoch, you feed data samples into the model in batches and run a gradient descent algorithm. This is one step in the training loop because you run the model in one forward pass (i.e., providing input and capturing output), and one backward pass (evaluating the loss metric from the output and deriving the gradient of each parameter all the way back to the input layer). The backward pass computes the gradient using automatic differentiation. Then, this gradient is used by the gradient descent algorithm to adjust the model parameters. There are multiple steps in one epoch.

Reusing the examples in Chapter 7, you can download the dataset[1] and split the dataset into two as follows:

```
import numpy as np
import torch

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]
```

Listing 8.1: Loading data from a CSV file

This dataset is small — only 768 samples. Here, it takes the first 700 as the training set and the rest as the test set.

It is not the focus of this chapter, but you can reuse the model, the loss function, and the optimizer from a previous chapter:

```
import torch.nn as nn
import torch.optim as optim
```

[1] https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

```
model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 8.2: Setting up a model, the loss function, and the optimizer

With the data and the model, this is the minimal training loop, with the forward and backward pass in each step:

```
n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
```

Listing 8.3: Training the model in a loop

In the inner for-loop, you take each batch in the dataset and evaluate the loss. The `loss` is a PyTorch tensor that remembers how it comes up with its value. Then you zero out all gradients that the optimizer manages and call `loss.backward()` to run the backpropagation algorithm. The result sets up the gradients of all the tensors that the tensor `loss` depends on directly and indirectly. Afterward, upon calling `step()`, the optimizer will check each parameter that it manages and update them.

After everything is done, you can run the model with the test set to evaluate its performance. The evaluation can be based on a different function than the loss function. For example, this classification problem uses accuracy:

```
...

# evaluate trained model with test set
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print("Accuracy {:.2f}".format(accuracy * 100))
```

Listing 8.4: Evaluating the trained model

Putting everything together, this is the complete code:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
```

```
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()

# evaluate trained model with test set
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print("Accuracy {:.2f}".format(accuracy * 100))
```

Listing 8.5: Creating, training, and evaluating a model

8.2 Collecting Statistics During Training

The training loop above should work well with small models that can finish training in a few seconds. But for a larger model or a larger dataset, you will find that it takes significantly longer to train. While you're waiting for the training to complete, you may want to see how it's going as you may want to interrupt the training if any mistake is made.

Usually, during training, you would like to see the following:

- ▷ In each step, you would like to know the loss metrics, and you are expecting the loss to go down
- ▷ In each step, you would like to know other metrics, such as accuracy on the training set, that are of interest but not involved in the gradient descent
- ▷ At the end of each epoch, you would like to evaluate the partially-trained model with the test set and report the evaluation metric
- ▷ At the end of the training, you would like to be above to visualize the above metrics

These all are possible, but you need to add more code into the training loop, as follows:

```
n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
```

```
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        acc = (y_pred.round() == ybatch).float().mean()
        # store metrics
        train_loss.append(float(loss))
        train_acc.append(float(acc))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        print(f"epoch {epoch} step {i} loss {loss} accuracy {acc}")
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

Listing 8.6: The training loop expanded to collect more statistics

As you collect the loss and accuracy in the list, you can plot them using matplotlib. But be careful that you collected training set statistics at each step, whereas the test set accuracy only at the end of the epoch. Thus you would like to show the *average accuracy* from the training loop in each epoch, so they are comparable to each other.

```
import matplotlib.pyplot as plt

# Plot the loss metrics, set the y-axis to start from 0
plt.plot(train_loss)
plt.xlabel("steps")
plt.ylabel("loss")
plt.ylim(0)
plt.show()

# plot the accuracy metrics
avg_train_acc = []
for i in range(n_epochs):
    start = i * batch_size
    average = sum(train_acc[start:start+batches_per_epoch]) / batches_per_epoch
    avg_train_acc.append(average)

plt.plot(avg_train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.ylim(0)
plt.show()
```

Listing 8.7: Visualizing accuracy metric during the training of a model

Putting everything together, below is the complete code:

```
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        # take a batch
        start = i * batch_size
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        acc = (y_pred.round() == ybatch).float().mean()
        # store metrics
```

```
        train_loss.append(float(loss))
        train_acc.append(float(acc))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        print(f"epoch {epoch} step {i} loss {loss} accuracy {acc}")
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")

# Plot the loss metrics
plt.plot(train_loss)
plt.xlabel("steps")
plt.ylabel("loss")
plt.ylim(0)
plt.show()

# plot the accuracy metrics
avg_train_acc = []
for i in range(n_epochs):
    start = i * batch_size
    average = sum(train_acc[start:start+batches_per_epoch]) / batches_per_epoch
    avg_train_acc.append(average)

plt.plot(avg_train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.ylim(0)
plt.show()
```

Listing 8.8: Complete code to build and train a model with visualized accuracy metric

The story does not end here. Indeed, you can add more code to the training loop, especially in dealing with a more complex model. One example is checkpointing. You may want to save your model (e.g., using pickle) so that, if for any reason, your program stops, you can restart the training loop from the middle. Another example is early stopping, which lets you monitor the accuracy you obtained with the test set at the end of each epoch and interrupt the training if you don't see the model improving for a while. This is because you probably can't go further, given the design of the model, and you do not want to overfit. Chapter 21 will cover more on this topic.

8.3 Using `tqdm` to Report the Training Progress

If you run the above code, you will find that there are a lot of lines printed on the screen while the training loop is running. Your screen may be cluttered. And you may also want to

see an animated progress bar to better tell you how far you are in the training progress. The library `tqdm` is the popular tool for creating the progress bar. Converting the above code to use `tqdm` cannot be easier:

```
for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

Listing 8.9: A training loop with `tqdm` progress bar

The usage of `tqdm` creates an iterator using `trange()` just like Python's `range()` function, and you can read the number in a loop. You can access the progress bar by updating its description or "postfix" data, but you have to do that before it exhausts its content. The `set_postfix()` function is powerful as it can show you anything.

In fact, there is a `tqdm()` function besides `trange()` that iterates over an existing list. You may find it easier to use, and you can rewrite the above loop as follows:

```
starts = [i*batch_size for i in range(batches_per_epoch)]

for epoch in range(n_epochs):
    with tqdm.tqdm(starts, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            Xbatch = Xtrain[start:start+batch_size]
```

```
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

Listing 8.10: Using `tqdm` to iterate over a list

The following is the complete code (without the matplotlib plotting):

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
```

```
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")
```

Listing 8.11: Complete code to train a model with progress printed using `tqdm`

8.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Software

tqdm.
https://github.com/tqdm/tqdm

Articles

torch.optim. PyTorch API.
https://pytorch.org/docs/stable/optim.html
Optimizing Model Parameters. PyTorch tutorials.
https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

8.5 Summary

In this chapter, you looked in detail at how to properly set up a training loop for a PyTorch model. In particular, you saw:

- ▷ What are the elements needed to implement in a training loop
- ▷ How a training loop connects the training data to the gradient descent optimizer
- ▷ How to collect information in the training loop and display them

In the next chapter, you will learn how to evaluate and compare deep learning models.

Evaluating PyTorch Models

9

Designing a deep learning model is sometimes an art. There are a lot of decision points, and it is not easy to tell what is the best. One way to come up with a design is by trial and error and evaluating the result on real data. Therefore, it is important to have a scientific method to evaluate the performance of your neural network and deep learning models. In fact, it is also the same method to compare any kind of machine learning models on a particular usage.

In this chapter, you will discover the received workflow to robustly evaluate model performance. We use PyTorch to build our models as examples, but the method can also be applied to other models. After completing this chapter, you will know:

- ▷ How to evaluate a PyTorch model using a verification dataset
- ▷ How to evaluate a PyTorch model with k-fold cross-validation

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Empirical Evaluation of Models
- ▷ Data Splitting
- ▷ Training a PyTorch Model with Validation
- ▷ k-Fold Cross-Validation

9.1 Empirical Evaluation of Models

In designing and configuring a deep learning model from scratch, there are a lot of decisions to make. This includes design decisions such as how many layers to use in a deep learning model, how big is each layer, and what kind of layers or activation functions to use. It can also be the choice of the loss function, optimization algorithm, number of epochs to train, and the interpretation of the model output. Luckily, sometimes, you can copy the structure of other people's networks. Sometimes, you can just make up your choice using some heuristics.

To tell if you made a good choice or not, the best way is to compare multiple alternatives by empirically evaluating them with actual data.

Deep learning is often used on problems that have very large datasets. That is tens of thousands or hundreds of thousands of data samples. This provides ample data for testing. But you need to have a robust test strategy to estimate the performance of your model on unseen data. Based on that, you can have a metric to compare among different model configurations.

9.2 Data Splitting

If you have a dataset of tens of thousands of samples or even more, you don't always need to give everything to your model for training. This will unnecessarily increase the complexity and lengthen the training time. More is not always better. You may not get the best result.

When you have a large amount of data, you should take a portion of it as the *training set* that is fed into the model for training. Another portion is kept as a *test set* to hold back from the training but verified with a trained or partially trained model as an evaluation. This step is usually called "train-test split."

Let's consider the Pima Indians Diabetes dataset[1]. You can load the data using NumPy:

```
import numpy as np
data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
```

Listing 9.1: Loading a dataset from CSV file

There are 768 data samples. It is not a lot but is enough to demonstrate the split. Let's consider the first 66% as the training set and the remaining as the test set. The easiest way to do so is by slicing an array:

```
# find the boundary at 66% of total samples
count = len(data)
n_train = int(count * 0.66)
# split the data at the boundary
train_data = data[:n_train]
test_data = data[n_train:]
```

Listing 9.2: Splitting the dataset into training and test sets

The choice of 66% is arbitrary, but you do not want the training set too small. Sometimes you may use 70%–30% split. But if the dataset is huge, you may even use 30%–70% split if 30% of training data is large enough.

If you split the data in this way, you're suggesting the datasets are shuffled so that the training set and the test set are equally diverse. If you find the original dataset is sorted and take the test set only at the end, you may find you have all the test data belonging to the same class or carrying the same value in one of the input features. That's not ideal.

Of course, you can call `np.random.shuffle(data)` before the split to avoid that. But many machine learning engineers usually use scikit-learn for this. See this example:

[1] https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv

```
import numpy as np
from sklearn.model_selection import train_test_split

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
train_data, test_data = train_test_split(data, test_size=0.33)
```

Listing 9.3: Splitting dataset using scikit-learn

But more commonly, it is done after you separate the input feature and output labels. Note that this function from scikit-learn can work not only on NumPy arrays but also on PyTorch tensors:

```
import numpy as np
import torch
from sklearn.model_selection import train_test_split

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
X = data[:, 0:8]
y = data[:, 8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
```

Listing 9.4: Splitting PyTorch tensors using scikit-learn

9.3 Training a PyTorch Model with Validation

Let's revisit the code for building and training a deep learning model on this dataset:

```
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

...

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross-entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(X_train) // batch_size
```

```
for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss)
            )
```

Listing 9.5: Building and training a PyTorch model

In this code, one batch is extracted from the training set in each iteration and sent to the model in the forward pass. Then you compute the gradient in the backward pass and update the weights.

While, in this case, you used binary cross-entropy as the loss metric in the training loop, you may be more concerned with the prediction accuracy. Calculating accuracy is easy. You round off the output (in the range of 0 to 1) to the nearest integer so you can get a binary value of 0 or 1. Then you count how much percentage your prediction matched the label; this gives you the accuracy.

But what is your prediction? It is `y_pred` above, which is the prediction by your current model on `X_batch`. Adding accuracy to the training loop becomes this:

```
for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
```

```
            # print progress, with accuracy
            acc = (y_pred.round() == y_batch).float().mean()
            bar.set_postfix(
                loss=float(loss)
                acc=float(acc)
            )
```

Listing 9.6: Training loop with accuracy metric reported

However, the `X_batch` and `y_batch` is used by the optimizer, and the optimizer will fine-tune your model so that it can predict `y_batch` from `X_batch`. And now you're using accuracy to check if `y_pred` match with `y_batch`. It is like cheating because if your model somehow remembers the solution, it can just report to you the `y_pred` and get perfect accuracy without actually inferring `y_pred` from `X_batch`.

Indeed, a deep learning model can be so convoluted that you cannot know if your model simply remembers the answer or is inferring the answer. Therefore, the best way is *not* to calculate accuracy from `X_batch` or anything from `X_train` but from something else: your test set. Let's add an accuracy measurement *after* each epoch using `X_test`:

```
for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            acc = (y_pred.round() == y_batch).float().mean()
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
    # evaluate model at end of epoch
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    acc = float(acc)
    print(f"End of {epoch}, accuracy {acc}")
```

Listing 9.7: Training loop with accuracy measurement after each epoch

In this case, the `acc` in the inner for-loop is just a metric showing the progress. Not much difference in displaying the loss metric, except it is not involved in the gradient descent algorithm. And you expect the accuracy to improve as the loss metric also improves.

In the outer for-loop, at the end of each epoch, you calculate the accuracy from `X_test`. The workflow is similar: You give the test set to the model and ask for its prediction, then count the number of matched results with your test set labels. But this accuracy is the one you should care about. It should improve as the training progresses, but if you do not see it improve (i.e., accuracy increase) or even deteriorates, you have to interrupt the training as it seems to start overfitting. Overfitting is when the model started to remember the training set rather than learning to infer the prediction from it. A sign of that is the accuracy from the training set keeps increasing while the accuracy from the test set decreases.

The following is the complete code to implement everything above, from data splitting to validation using the test set:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import train_test_split

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
X = data[:, 0:8]
y = data[:, 8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)

# loss function and optimizer
loss_fn = nn.BCELoss()  # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(X_train) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
```

```
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            acc = (y_pred.round() == y_batch).float().mean()
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
    # evaluate model at end of epoch
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    acc = float(acc)
    print(f"End of {epoch}, accuracy {acc}")
```

Listing 9.8: Complete code to train a model with evaluation using a test set

The code above will print the following:

```
End of 0, accuracy 0.5787401795387268
End of 1, accuracy 0.6102362275123596
End of 2, accuracy 0.6220472455024719
End of 3, accuracy 0.6220472455024719
...
End of 17, accuracy 0.6811023354530334
End of 18, accuracy 0.6850393414497375
End of 19, accuracy 0.6889764070510864
End of 20, accuracy 0.6850393414497375
End of 21, accuracy 0.6889764070510864
End of 22, accuracy 0.6889764070510864
End of 23, accuracy 0.6889764070510864
End of 24, accuracy 0.6889764070510864
End of 25, accuracy 0.6850393414497375
End of 26, accuracy 0.6811023354530334
End of 27, accuracy 0.6771653294563293
End of 28, accuracy 0.6771653294563293
End of 29, accuracy 0.6692913174629211
End of 30, accuracy 0.6732283234596252
End of 31, accuracy 0.6692913174629211
End of 32, accuracy 0.6692913174629211
End of 33, accuracy 0.6732283234596252
End of 34, accuracy 0.6771653294563293
End of 35, accuracy 0.6811023354530334
End of 36, accuracy 0.6811023354530334
End of 37, accuracy 0.6811023354530334
End of 38, accuracy 0.6811023354530334
End of 39, accuracy 0.6811023354530334
End of 40, accuracy 0.6811023354530334
End of 41, accuracy 0.6771653294563293
End of 42, accuracy 0.6771653294563293
```

```
End of 43, accuracy 0.6771653294563293
End of 44, accuracy 0.6771653294563293
End of 45, accuracy 0.6771653294563293
End of 46, accuracy 0.6771653294563293
End of 47, accuracy 0.6732283234596252
End of 48, accuracy 0.6732283234596252
End of 49, accuracy 0.6732283234596252
```

Output 9.1: Accuracy metric on the test set printed from the training loop

9.4 *k*-Fold Cross-Validation

In the above example, you calculated the accuracy from the test set. It is used as a *score* for the model as you progressed in the training. You want to stop at the point where this score is at its maximum. In fact, by merely comparing the score from this test set, you know your model works best after epoch 21 and start to overfit afterwards. Is that right?

If you built two models of different designs, should you just compare these models' accuracy on the same test set and claim one is better than another?

You have a test set to evaluate your model. But you may argue that, by chance, one model fits better to this particular test set but not always better. To make a stronger argument on which model is better independent of the selection of the test set, you can try *multiple test sets* and average the accuracy.

This is what a k-fold cross-validation does. It is a procedure to decide on which *design* works better. It works by repeating the training process from scratch for k times, each with a different composition of the training and test set. Because of that, you will have k models and k accuracy scores from their respective test set. You are not only interested in the average accuracy, but also the standard deviation. The standard deviation tells whether the accuracy score is consistent or some test set is particularly good or bad to a model.

Since k-fold cross-validation is to train the model from scratch a few times, it is best to wrap around the training loop in a function:

```
def model_train(X_train, y_train, X_test, y_test):
    # create new model
    model = nn.Sequential(
        nn.Linear(8, 12),
        nn.ReLU(),
        nn.Linear(12, 8),
        nn.ReLU(),
        nn.Linear(8, 1),
        nn.Sigmoid()
    )

    # loss function and optimizer
    loss_fn = nn.BCELoss()  # binary cross-entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 25    # number of epochs to run
```

```
    batch_size = 10  # size of each batch
    batches_per_epoch = len(X_train) // batch_size

    for epoch in range(n_epochs):
        with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0, disable=True
                         ) as bar:
            bar.set_description(f"Epoch {epoch}")
            for i in bar:
                # take a batch
                start = i * batch_size
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
    # evaluate accuracy at end of training
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    return float(acc)
```

Listing 9.9: A function to create a PyTorch model

The code above is deliberately not printing anything (with `disable=True` in `tqdm`) to keep the screen less cluttered.

Also from scikit-learn, you have a function for k-fold cross-validation. You can make use of it to produce a robust estimate of model accuracy:

```
from sklearn.model_selection import StratifiedKFold

# define 5-fold cross-validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    acc = model_train(X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    cv_scores.append(acc)
# evaluate the model
print("%.2f%% (+/- %.2f%%)" % (np.mean(cv_scores)*100, np.std(cv_scores)*100))
```

Listing 9.10: k-fold cross-validation on a PyTorch model

Running this prints:

```
Accuracy: 0.64
Accuracy: 0.67
Accuracy: 0.68
Accuracy: 0.63
Accuracy: 0.59
64.05% (+/- 3.30%)
```

Output 9.2: Result from k-fold cross-validation

In scikit-learn, there are multiple k-fold cross-validation functions, and the one used here is stratified k-fold. It assumes `y` are class labels and takes into account of their values such that it will provide a balanced class representation in the splits.

The code above used $k = 5$ or five splits. It means splitting the dataset into five equal portions, picking one of them as the test set and combining the rest into a training set. There are five ways of doing that, so the for-loop above will have five iterations. In each iteration, you call the `model_train()` function and obtain the accuracy score in return. Then you save it into a list, which will be used to calculate the mean and standard deviation at the end.

The `kfold` object will return to you the *indices*. Hence you do not need to run the train-test split in advance but use the indices provided to extract the training set and test set on the fly when you call the `model_train()` function.

The result above shows the model is moderately good, at 64% average accuracy. And this score is stable since the standard deviation is at 3%. This means that most of the time, you expect the model accuracy to be 61% to 67%. You may try to change the model above, such as adding or removing a layer, and see how much change you have in the mean and standard deviation. You may also try to increase the number of epochs used in training and observe the result.

The mean and standard deviation from the k-fold cross-validation is what you should use to benchmark a model design.

Tying it all together, below is the complete code for k-fold cross-validation:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import StratifiedKFold

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
X = data[:, 0:8]
y = data[:, 8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

def model_train(X_train, y_train, X_test, y_test):
    # create new model
    model = nn.Sequential(
        nn.Linear(8, 12),
        nn.ReLU(),
```

```
        nn.Linear(12, 8),
        nn.ReLU(),
        nn.Linear(8, 1),
        nn.Sigmoid()
    )

    # loss function and optimizer
    loss_fn = nn.BCELoss()  # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 25    # number of epochs to run
    batch_size = 10  # size of each batch
    batches_per_epoch = len(X_train) // batch_size

    for epoch in range(n_epochs):
        with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0, disable=True
                         ) as bar:
            bar.set_description(f"Epoch {epoch}")
            for i in bar:
                # take a batch
                start = i * batch_size
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
    # evaluate accuracy at end of training
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    return float(acc)

# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    acc = model_train(X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    cv_scores.append(acc)
# evaluate the model
print("%.2f%% (+/- %.2f%%)" % (np.mean(cv_scores)*100, np.std(cv_scores)*100))
```

Listing 9.11: Evaluating a model using k-fold cross-validation

9.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Stratified K-Folds cross-validator. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

KFold cross-validator. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

9.6 Summary

In this chapter, you discovered the importance of having a robust way to estimate the performance of your deep learning models on unseen data, and you learned how to do that. You saw:

- ▷ How to split data into training and test sets using scikit-learn
- ▷ How to do k-fold cross-validation with the help of scikit-learn
- ▷ How to modify the training loop in a PyTorch model to incorporate test set validation and cross-validation

In the next few chapters, you will put everything you learned so far into practice by finishing some projects.

Project: Building a Multiclass Classification Model in PyTorch

10

The PyTorch library is for deep learning. Some applications of deep learning models are used to solve regression or classification problems. In this chapter, you will discover how to use PyTorch to develop and evaluate neural network models for multiclass classification problems.

After completing this step-by-step tutorial, you will know:

- ▷ How to load data from CSV and make it available to PyTorch
- ▷ How to prepare multiclass classification data for modeling with neural networks
- ▷ How to use cross-validation to evaluate a PyTorch neural network model

Let's get started.

10.1 Problem Description

In this chapter, you will use a standard machine learning dataset called the iris flowers dataset. It is a well-studied dataset and good for practicing machine learning. It has four input variables; all are numeric and length measurements in centimeters. Therefore, they are on a similar scale. Each data sample describes the properties of an observed iris flower. Its goal is to use the measurements (input features) to classify the iris species (output label).

There are three iris species in the dataset. Therefore, it is a multiclass classification problem. Multiclass classification problems are special because they require special handling to specify a class.

This dataset came from Sir Ronald Fisher, the father of modern statistics. It is the best-known dataset for pattern recognition, and you can achieve a model accuracy in the range of 95% to 97%. You can make this your target in developing the deep learning model.

You can download the iris flowers dataset from the UCI Machine Learning repository and place it in your current working directory with the filename "`iris.csv`". You can also download the dataset here:

- ▷ `https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv`

10.2 Load the Dataset

There are multiple ways to read a CSV file. The easiest way is probably to use a pandas library. After reading the dataset, you want to split it into features and labels as you need to further process the labels before use. Unlike NumPy or PyTorch tensors, pandas DataFrame can do slicing by indices only through `iloc`:

```
import pandas as pd
data = pd.read_csv("iris.csv", header=None)
X = data.iloc[:, 0:4]
y = data.iloc[:, 4:]
```

Listing 10.1: Loading a dataset from CSV file

Now, you have loaded the dataset and split the attributes (i.e., input features, columns in the DataFrame) as `X` and the output variables (i.e., species labels) as a single-column DataFrame `y`.

10.3 Encode the Categorical Variable

The species labels are strings, but you want them in numbers. It is because numerical data are easier to use. In this dataset, the three class labels are `Iris-setosa`, `Iris-versicolor`, and `Iris-virginica`. One way to convert these labels into a number (i.e., encode them) is simply to assign an integer value such as 0, 1, or 2 to replace these labels. But there is a problem: You do not want the model to think that `Iris-virginica` it is the sum of `Iris-setosa` and `Iris-versicolor`. In fact, in statistics, there are levels of measurement:

- ▷ Nominal numbers: Those numbers are, in fact, names. Operations on them do not make sense
- ▷ Ordinal numbers: They are orders of something. Comparing for greater or less than makes sense, but addition or subtraction does not
- ▷ Interval numbers: They are measurements, such as the year today, so that subtraction bears meaning (e.g., how old are you), but the zero value is arbitrary and not special
- ▷ Ratio numbers: Like interval, but the zero is meaningful, such as the measure of length or time. In this case, both subtraction and division bear meaning, and you can say that something is twice as long.

The encoded label is nominal. You do not want to mistake it as interval or ratio data, but your model would not know. One way to avoid such a mistake is with *one-hot encoding*, which instead of converting a label into an integer, converts a label into a *one-hot vector*. A one-hot vector is a vector of integers, but only one of them is 1 and the rest are all zero. In this case, you convert the labels into the following:

```
Iris-setosa      1   0   0
Iris-versicolor  0   1   0
Iris-virginica   0   0   1
```

Output 10.1: One-hot labels to the iris dataset

The above is a one-hot encoded binary matrix. You don't need to create it manually. You can encode the strings consistently to integers using the scikit-learn class `LabelEncoder` or into one-hot encoding vectors using the class `OneHotEncoder`:

```
...
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
print(ohe.categories_)

y = ohe.transform(y)
print(y)
```

Listing 10.2: Transforming textual labels into one-hot encoding

From these, you can see the `OneHotEncoder` learned the three categories:

```
[array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)]
```

Output 10.2: The categories as from `ohe.categories_`

and the string labels are transformed into one-hot vectors like the following:

```
[[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
...
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]]
```

Output 10.3: The one-hot encoded label in `y`

10.4 Define the Neural Network Model

Now you need to have a model that can take the input and predict the output, ideally in the form of one-hot vectors. There is no science behind the design of a perfect neural network model. But know one thing — it has to take in a vector of 4 features and output a vector of 3 values. The 4 features correspond to what you have in the dataset. The 3-value output is because we know the one-hot vector has 3 elements. Anything can be in between, known as the "hidden layers," since they are neither input nor output.

The simplest is to have only one hidden layer. Let's make one like this:

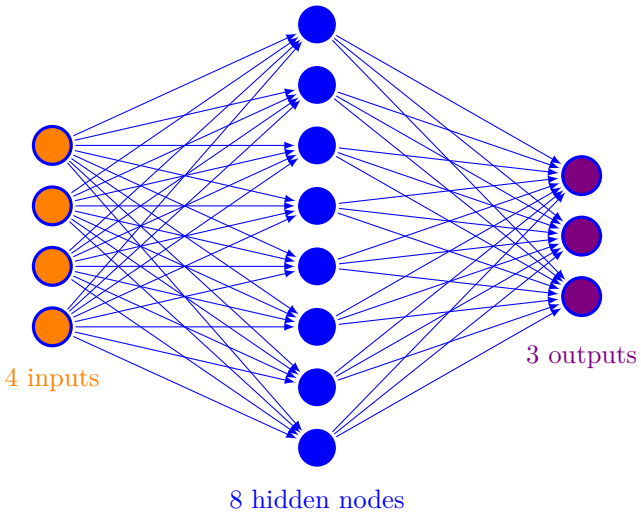


Figure 10.1: Example network structure

Such a design is called the network topology. You should use a *softmax* activation at the output layer. In the formula, it means:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{3} e^{z_j}}$$

This normalizes the values (z_1, z_2, z_3) and applies a nonlinear function such that the sum of all 3 outputs will be 1, and each of them is in the range of 0 to 1. This makes the output look like a vector of probabilities. The use of the softmax function at the output is the signature of a multiclass classification model. But in PyTorch, you can skip this if you combine it with an appropriate loss function.

In PyTorch, you can build such a model as follows:

```
import torch.nn as nn

class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.output(x)
        return x

model = Multiclass()
```

Listing 10.3: The model in PyTorch code

The output of this model is the *weight* of the three classes. Ideally, the model output will be such that only one of the elements is positive infinity, and the rest are negative infinity, making an extreme contrast with absolute confidence to which of the three classes the input

features belong. In an imperfect situation, as always happens, you can expect a good model to tell you that one of the values is very positive, and the others are very negative. Or if you transform these values using a sigmoid function or softmax function, one is very close to 1, and the others are very close to 0.

In this case, the loss metric for the output can simply be measuring how close the output is to the one-hot vector you transformed from the label. But usually, in multiclass classification, you use categorical cross-entropy as the loss metric. In the formula, it is:

$$H(p, q) = -\sum_x p(x) \log q(x)$$

This means, given the real probability vector $p(x)$ and the predicted probability vector $q(x)$, the similarity is the sum of the product of $p(x)$ and $\log q(x)$ for each element x. The one-hot vector is considered as the probability vector $p(x)$, and the model output is $q(x)$. Since it is a one-hot vector, only the actual class has $p(x) = 1$, and the other classes have $p(x) = 0$. The sum above is essentially $-\log q(x)$ of the actual class x. The value will be 0 when $q(x) = 1$, and as $q(x)$ approaches 0 (the minimum value softmax can produce), $-\log q(x)$ approaches infinity.

Below is how you can define the loss metric. The `CrossEntropyLoss` function in PyTorch combines the softmax function with the cross-entropy calculation, so you don't need any activation function at the output layer of your model. You also need an optimizer, and Adam is chosen below.

```
import torch.optim as optim

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 10.4: Loss function and optimizer

Note that when you define the optimizer, you need to tell it the model parameters as well since these are what the optimizer is going to update.

Now you need to run the training loop to train your model. Minimally, you need to put three steps in the loop: a forward pass, a backward pass, and the weight update. The forward pass provides the input to the model and takes the output. The backward pass starts with the loss metric, which is based on the model output, and propagates the gradient back to the input. The weight update is based on the gradient used to update the weights.

A minimal training loop can be implemented using a for-loop. But you can make use of `tqdm` to create a progress bar visualization:

```
import tqdm

# convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# training parameters
n_epochs = 200
batch_size = 5
```

```
batches_per_epoch = len(X) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
```

Listing 10.5: Training loop

10.5 Benchmark the Model

The goal of a model is never to match the dataset per se. The reason you want to build a machine learning model is to prepare for the data you will encounter in the future, which is unseen yet. How can you know the model can do that? You need a test set. It is a dataset that is the same structure as the one used in training but is separate. So it is like unseen data from the training process, and you can take that as a benchmark. This technique of evaluating a model is called *cross-validation*.

Usually, you do not add a test set but split the data set you obtained into a training set and a test set. Then you use the test set to evaluate the model at the end. Such a benchmark serves another purpose: You do not want your model to overfit. It means the model learned too much about the training set and failed to generalize. If this happens, you will see that the model does not give a good result on the test set.

Splitting the data into a training set and a test set can be easily done with scikit-learn. The workflow from loading data to one-hot encoding and splitting is as follows:

```
import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# read data and apply one-hot encoding
data = pd.read_csv("iris.csv", header=None)
X = data.iloc[:, 0:4]
y = data.iloc[:, 4:]
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
y = ohe.transform(y)
```

```
# convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
```

Listing 10.6: Splitting data into training and test sets

The arguments `train_size=0.7` with `shuffle=True` means to randomly select 70% of the samples in the dataset into the training set, while the rest will become the test set.

Once you do that, you need to modify the training loop to use the training set in training and use the test set at the end of each epoch for benchmarking:

```
n_epochs = 200
batch_size = 5
batches_per_epoch = len(X_train) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
    y_pred = model(X_test)
    ce = loss_fn(y_pred, y_test)
    acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
    print(f"Epoch {epoch} validation: Cross-entropy={float(ce)}, Accuracy={float(acc)}")
```

Listing 10.7: Benchmarking the model in the training loop

That's almost everything you need to finish a deep learning model in PyTorch. But you may also want to do a bit more: Firstly, after all the training epochs, you may want to roll back the model to the best you ever achieved rather than the last one. Secondly, you may want to produce a graph to visualize how the cross-entropy and accuracy progressed.

It is not difficult to do. In the training loop, you keep track of the accuracy on the test set and keep a copy of the model whenever this accuracy is higher. And at the same time, remember the metrics you calculated in a list. Then at the end of the training loop, you restore the best model you ever saw and plot the metrics as a time series.

In code, this is how the training loop is modified into:

```
import copy
import tqdm
import numpy as np

n_epochs = 200
batch_size = 5
batches_per_epoch = len(X_train) // batch_size

best_acc = - np.inf   # init to negative infinity
best_weights = None
train_loss_hist = []
train_acc_hist = []
test_loss_hist = []
test_acc_hist = []

for epoch in range(n_epochs):
    epoch_loss = []
    epoch_acc = []
    # set model in training mode and run through each batch
    model.train()
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # compute and store metrics
            acc = (torch.argmax(y_pred, 1) == torch.argmax(y_batch, 1)).float().mean()
            epoch_loss.append(float(loss))
            epoch_acc.append(float(acc))
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
    # set model in evaluation mode and run through the test set
    model.eval()
    y_pred = model(X_test)
    ce = loss_fn(y_pred, y_test)
    acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
    ce = float(ce)
    acc = float(acc)
    train_loss_hist.append(np.mean(epoch_loss))
    train_acc_hist.append(np.mean(epoch_acc))
    test_loss_hist.append(ce)
    test_acc_hist.append(acc)
```

```
    if acc > best_acc:
        best_acc = acc
        best_weights = copy.deepcopy(model.state_dict())
    print(f"Epoch {epoch} validation: Cross-entropy={ce}, Accuracy={acc}")

model.load_state_dict(best_weights)
```

Listing 10.8: The training loop with the evaluation metric collected and best model restored

When you run it, you will see something like this, where you see the accuracy and cross-entropy loss at each epoch:

```
Epoch 0: 100%|████████████████████| 21/21 [00:00<00:00, 850.82batch/s, acc=0.4, loss=1.23]
Epoch 0 validation: Cross-entropy=1.10, Accuracy=60.0%
Epoch 1: 100%|████████████████████| 21/21 [00:00<00:00, 3155.53batch/s, acc=0.4, loss=1.24]
Epoch 1 validation: Cross-entropy=1.08, Accuracy=57.8%
Epoch 2: 100%|████████████████████| 21/21 [00:00<00:00, 3489.58batch/s, acc=0.4, loss=1.24]
Epoch 2 validation: Cross-entropy=1.07, Accuracy=60.0%
Epoch 3: 100%|████████████████████| 21/21 [00:00<00:00, 3312.79batch/s, acc=0.4, loss=1.22]
Epoch 3 validation: Cross-entropy=1.06, Accuracy=62.2%
...
Epoch 197: 100%|████████████████████| 21/21 [00:00<00:00, 3529.57batch/s, acc=1, loss=0.563]
Epoch 197 validation: Cross-entropy=0.61, Accuracy=97.8%
Epoch 198: 100%|████████████████████| 21/21 [00:00<00:00, 3479.10batch/s, acc=1, loss=0.563]
Epoch 198 validation: Cross-entropy=0.61, Accuracy=97.8%
Epoch 199: 100%|████████████████████| 21/21 [00:00<00:00, 3337.52batch/s, acc=1, loss=0.563]
Epoch 199 validation: Cross-entropy=0.61, Accuracy=97.8%
```

Output 10.4: Output from the training loop

You added more lines into the training loop but for a good reason. It is a good practice to switch the model between training mode and evaluation mode when you are switching between the training set and test set. In this particular model, nothing is changed. But for some other models, it will affect the model's behavior.

You collected metrics in Python lists. You need to be careful to convert the PyTorch tensor (even if it is a scalar value) into a Python float. The purpose of this conversion is to make a copy of the number so that PyTorch will not silently mutate it (e.g., through the optimizer).

After each epoch, you calculated the accuracy based on the test set, and the model weight is stored if you saw the accuracy is higher. However, when you take out the model weight, you should make a deep copy; otherwise, you lost them when the model changes its weights in the next epoch.

Finally, you can plot the loss and accuracy across epochs using matplotlib as follows:

```
import matplotlib.pyplot as plt

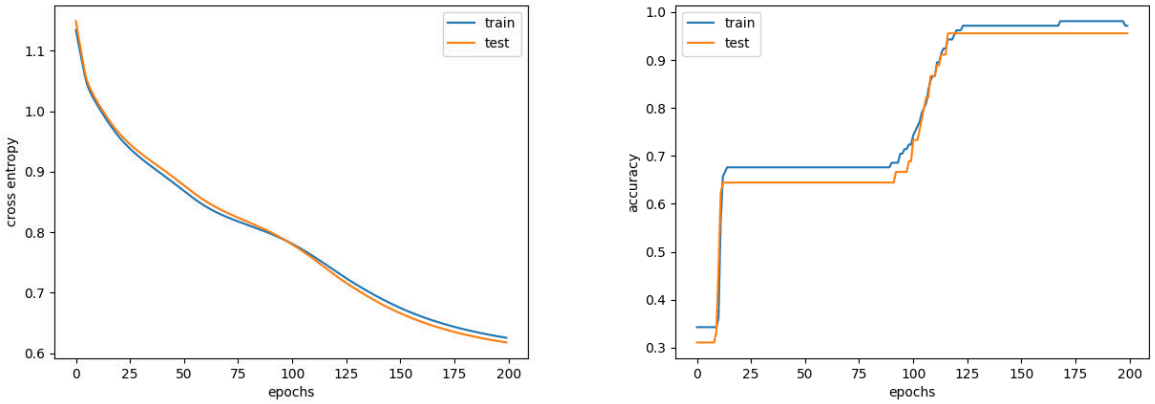
plt.plot(train_loss_hist, label="train")
plt.plot(test_loss_hist, label="test")
plt.xlabel("epochs")
plt.ylabel("cross entropy")
```

```
plt.legend()
plt.show()

plt.plot(train_acc_hist, label="train")
plt.plot(test_acc_hist, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.legend()
plt.show()
```

Listing 10.9: Plotting the loss and accuracy

A typical result is as follows:



(a) Loss *(b) Accuracy*

Figure 10.2: Training and validation loss and accuracy

You can see from the graph that at the beginning, both training and test accuracy are very low. This is when your model is underfitting and is performing horribly. As you keep training the model, the accuracy increases, and the cross-entropy loss decreases. But at a certain point, the training accuracy is higher than the test accuracy, and in fact, even when the training accuracy improves, the test accuracy flattened or even lowered. This is when the model overfitted, and you do not want to use such a model. That's why you want to keep track of the test accuracy and restore the model weight to the best result based on the test set.

10.6 Complete Example

Putting everything together, the following is the complete code:

```
import copy

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
```

```
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# read data and apply one-hot encoding
data = pd.read_csv("iris.csv", header=None)
X = data.iloc[:, 0:4]
y = data.iloc[:, 4:]
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
y = ohe.transform(y)

# convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.output(x)
        return x

# loss metric and optimizer
model = Multiclass()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 200
batch_size = 5
batches_per_epoch = len(X_train) // batch_size

best_acc = - np.inf   # init to negative infinity
best_weights = None
train_loss_hist = []
train_acc_hist = []
test_loss_hist = []
test_acc_hist = []

# training loop
for epoch in range(n_epochs):
    epoch_loss = []
    epoch_acc = []
    # set model in training mode and run through each batch
```

```
    model.train()
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # compute and store metrics
            acc = (torch.argmax(y_pred, 1) == torch.argmax(y_batch, 1)).float().mean()
            epoch_loss.append(float(loss))
            epoch_acc.append(float(acc))
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
    # set model in evaluation mode and run through the test set
    model.eval()
    y_pred = model(X_test)
    ce = loss_fn(y_pred, y_test)
    acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
    ce = float(ce)
    acc = float(acc)
    train_loss_hist.append(np.mean(epoch_loss))
    train_acc_hist.append(np.mean(epoch_acc))
    test_loss_hist.append(ce)
    test_acc_hist.append(acc)
    if acc > best_acc:
        best_acc = acc
        best_weights = copy.deepcopy(model.state_dict())
    print(f"Epoch {epoch} validation: Cross-entropy={ce:.2f}, Accuracy={acc*100:.1f}%")

# Restore best model
model.load_state_dict(best_weights)

# Plot the loss and accuracy
plt.plot(train_loss_hist, label="train")
plt.plot(test_loss_hist, label="test")
plt.xlabel("epochs")
plt.ylabel("cross entropy")
plt.legend()
plt.show()

plt.plot(train_acc_hist, label="train")
plt.plot(test_acc_hist, label="test")
plt.xlabel("epochs")
```

```
plt.ylabel("accuracy")
plt.legend()
plt.show()
```

Listing 10.10: An example of multiclass classification in PyTorch

10.7 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Iris Data Set. UCI Machine Learning Repository.
https://archive.ics.uci.edu/ml/datasets/Iris

Softmax. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.Softmax.html

CrossEntropyLoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

One-Hot encoder. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

10.8 Summary

In this chapter, you discovered how to develop and evaluate a neural network for multiclass classification using PyTorch.

By completing this chapter, you learned:

- ▷ How to load data and convert them to PyTorch tensors
- ▷ How to prepare multiclass classification data for modeling using one-hot encoding
- ▷ How to define a deep learning model with PyTorch for multiclass classification
- ▷ How to measure the likeness of model output to the expected result of a multiclass classification
- ▷ How to run a training loop on a PyTorch model and collect evaluation metrics

In the next chapter, you will do another classification project.

Project: Building a Binary Classification Model in PyTorch

11

PyTorch library is for deep learning. Some applications of deep learning models are to solve regression or classification problems. In this chapter, you will discover how to use PyTorch to develop and evaluate neural network models for binary classification problems.

After completing this chapter, you will know:

- ▷ How to load training data and make it available to PyTorch
- ▷ How to design and train a neural network
- ▷ How to evaluate the performance of a neural network model using k-fold cross-validation
- ▷ How to run a model in inference mode
- ▷ How to create receiver operating characteristics curve for a binary classification model

Let's get started.

11.1 Description of the Dataset

The dataset you will use in this chapter is the Sonar dataset.

This is a dataset that describes sonar chirp returns bouncing off different services. The 60 input variables are the strength of the returns at different angles. It is a binary classification problem that requires a model to differentiate rocks from metal cylinders.

You can learn more about this dataset on the UCI Machine Learning repository. You can download the dataset for free and place it in your working directory with the filename `sonar.csv`:

- ▷ https://archive.ics.uci.edu/ml/machine-learning-databases/undocumented/connectionist-bench/sonar/sonar.all-data

It is a well-understood dataset. All the variables are continuous and generally in the range of 0 to 1. The output variable is a string "M" for mine and "R" for rock, which will need to be converted to integers 1 and 0.

A benefit of using this dataset is that it is a standard benchmark problem. This means that we have some idea of the expected skill of a good model. Using cross-validation, a neural network should be able to achieve a performance[1] of 84% to 88% accuracy.

11.2 Load the Dataset

If you have downloaded the dataset in CSV format and saved it as `sonar.csv` in the local directory, you can load the dataset using pandas. There are 60 input variables (`X`) and one output variable (`y`). Because the file contains mixed data of strings and numbers, it is easier to read them using pandas rather than other tools such as NumPy.

Data can be read as follows:

```
import pandas as pd

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]
```

Listing 11.1: Loading the dataset

It is a binary classification dataset. You would prefer a numeric label over a string label. You can do such conversion with `LabelEncoder` in scikit-learn. The `LabelEncoder` is to map each label to an integer. In this case, there are only two labels and they will become 0 and 1.

Using it, you need to first call the `fit()` function to make it learn what labels are available. Then call `transform()` to do the actual conversion. Below is how you use `LabelEncoder` to convert `y` from strings into 0 and 1:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
```

Listing 11.2: Encode the dataset label

You can see the labels using:

```
print(encoder.classes_)
```

Listing 11.3: Show the classes as from the CSV file

which outputs:

```
['M' 'R']
```

Output 11.1: The class labels from the CSV file

and if you run `print(y)`, you would see the following

[1] http://www.is.umk.pl/projects/datasets.html#Sonar

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

Output 11.2: The integer-encoded class labels

You see the labels are converted into 0 and 1. From the `encoder.classes_`, you know that 0 means "M" and 1 means "R". They are also called the negative and positive classes respectively in the context of binary classification.

Afterward, you should convert them into PyTorch tensors as this is the format a PyTorch model would like to work with.

```
import torch

X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
```

Listing 11.4: Converting data into PyTorch tensors

11.3 Creating a Model

Now you're ready for the neural network model.

As you have seen in some previous chapters, the easiest neural network model is a 3-layer model that has only one hidden layer. A deep learning model is usually referring to those with more than one hidden layer. All neural network models have parameters called weights. The more parameters a model has, heuristically we believe that it is more powerful. Should you use a model with fewer layers but more parameters on each layer, or a model with more layers but fewer parameters each? Let's find out.

A model with more parameters on each layer is called a wider model. In this example, the input data has 60 features to predict one binary variable. You can assume to make a wide model with one hidden layer of 180 neurons (three times the input features). Such model can be built using PyTorch:

```
import torch.nn as nn

class Wide(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(60, 180)
        self.relu = nn.ReLU()
        self.output = nn.Linear(180, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.hidden(x))
```

```
        x = self.sigmoid(self.output(x))
        return x
```

Listing 11.5: Building a wide model

Because it is a binary classification problem, the output have to be a vector of length 1. Then you also want the output to be between 0 and 1 so you can consider that as probability or the model's confidence of prediction that the input corresponds to the "positive" class.

A model with more layer is called a deeper model. Considering that the previous model has one layer with 180 neurons, you can try one with three layers of 60 neurons each instead. Such model can be built using PyTorch:

```
import torch.nn as nn

class Deep(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.sigmoid(self.output(x))
        return x
```

Listing 11.6: Building a deep model

You can confirm that these two models are having similar number of parameters, as follows:

```
# Compare model sizes
model1 = Wide()
model2 = Deep()
print(sum([x.reshape(-1).shape[0] for x in model1.parameters()]))  # 11161
print(sum([x.reshape(-1).shape[0] for x in model2.parameters()]))  # 11041
```

Listing 11.7: Comparing the size of the two models

There will be all the model's parameters returned by `model1.parameters()` and each is a PyTorch tensor. Then you can reformat each tensor into a vector and count the length of the vector, using `x.reshape(-1).shape[0]`. So the above sum up the total number of parameters in each model.

11.4 Comparing Models with Cross-Validation

Should you use a wide model or a deep model? One way to tell is to use cross-validation to compare them.

It is a technique that, use a *training set* of data to train the model and then use a *test set* of data to see how accurate the model can predict. The result from test set is what you should focus on. But you do not want to test a model once because if you see an extremely good or bad result, it may be by chance. You want to run this process k times with different training and test sets, such that you are ensured that you are comparing the *model design*, not the result of a particular training.

The technique that you can use here is called k-fold cross-validation (more in Chapter 19). It is to split a larger dataset into k portions and take one portion as the test set while the $k-1$ portions are combined as the training set. There are k different such combinations. Therefore you can repeat the experiment for k times and take the average result.

In scikit-learn, you have a function for stratified k-fold. Stratified means that when the data is split into k portions, the algorithm will look at the labels (i.e., the positive and negative classes in a binary classification problem) to ensure it is split in such a way that each portion contains equal number of either classes.

Running k-fold cross-validation is trivial, such as the following:

```
# define 5-fold cross-validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores.append(acc)

# evaluate the model
acc = np.mean(cv_scores)
std = np.std(cv_scores)
print("Model accuracy: %.2f%% (+/- %.2f%%)" % (acc*100, std*100))
```

Listing 11.8: Running k-fold cross-validation

Simply speaking, you use `StratifiedKFold()` from scikit-learn to split the dataset. This function returns to you the indices. Hence you can create the split dataset using `X[train]` and `X[test]` and named them training set and validation set (so it is not confused with "test set" which will be used later, after we picked our model design). You assume to have a function that runs the training loop on a model and give you the accuracy on the validation set. You can then find the mean and standard deviation of this score as the performance metric of such model design. Note that you need to create a new model every time in the for-loop above because you should not re-train a trained model in the k-fold cross-validation.

The training loop can be defined as follows:

```python
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

def model_train(model, X_train, y_train, X_val, y_val):
    # loss function and optimizer
    loss_fn = nn.BCELoss()  # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 250   # number of epochs to run
    batch_size = 10  # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_acc = - np.inf   # init to negative infinity
    best_weights = None

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
        # evaluate accuracy at end of each epoch
        model.eval()
        y_pred = model(X_val)
        acc = (y_pred.round() == y_val).float().mean()
        acc = float(acc)
        if acc > best_acc:
            best_acc = acc
            best_weights = copy.deepcopy(model.state_dict())
    # restore model and return best accuracy
    model.load_state_dict(best_weights)
    return best_acc
```

Listing 11.9: Function to train a PyTorch model with the given dataset

The training loop above contains the usual elements: The forward pass, the backward pass, and the gradient descent weight updates. But it is extended to have an evaluation step after each epoch: You run the model at evaluation mode and check how the model predicts the *validation set*. The accuracy on the validation set is remembered along with the model weight. At the end of the training, the best weight is restored to the model and the best accuracy is returned. This returned value is the best you ever encountered during the many epochs of training and it is based on the validation set.

Note that you set `disable=True` in the `tqdm` above. You can set it to `False` to see the training set loss and accuracy as you progress in the training.

Remind that the goal is to pick the best design and train the model again, which in the training, you want to have an evaluation score so you know what to expect in production. Thus you should split the entire dataset you obtained into a training set and test set. Then you further split the training set in k-fold cross-validation.

With these, here is how you can compare the two model designs: By running k-fold cross-validation on each and compare the accuracy:

```
from sklearn.model_selection import StratifiedKFold, train_test_split

# train-test split: Hold out the test set for final model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# define 5-fold cross-validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores_wide = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores_wide.append(acc)
cv_scores_deep = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Deep()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (deep): %.2f" % acc)
    cv_scores_deep.append(acc)

# evaluate the model
wide_acc = np.mean(cv_scores_wide)
wide_std = np.std(cv_scores_wide)
deep_acc = np.mean(cv_scores_deep)
deep_std = np.std(cv_scores_deep)
print("Wide: %.2f%% (+/- %.2f%%)" % (wide_acc*100, wide_std*100))
print("Deep: %.2f%% (+/- %.2f%%)" % (deep_acc*100, deep_std*100))
```

Listing 11.10: Compare models with k-fold cross-validation

You may see the output of above as follows:

```
Accuracy (wide): 0.72
Accuracy (wide): 0.66
Accuracy (wide): 0.83
Accuracy (wide): 0.76
Accuracy (wide): 0.83
Accuracy (deep): 0.90
Accuracy (deep): 0.72
Accuracy (deep): 0.93
Accuracy (deep): 0.69
Accuracy (deep): 0.76
Wide: 75.86% (+/- 6.54%)
Deep: 80.00% (+/- 9.61%)
```

Output 11.3: Result of the k-fold cross-validation

So you found that the deeper model is better than the wider model, in the sense that the mean accuracy is higher.

11.5 Retrain the Final Model

Now you know which design to pick, you want to rebuild the model and retrain it. Usually in k-fold cross-validation, you will use a smaller dataset to make the training faster. The final accuracy is not an issue because the gold of k-fold cross-validation is to tell which design is better. In the final model, you want to provide more data and produce a better model, since this is what you will use in production.

As you already split the data into training and test set, these are what you will use. In Python code,

```
# rebuild model with full set of training data
if wide_acc > deep_acc:
    print("Retrain a wide model")
    model = Wide()
else:
    print("Retrain a deep model")
    model = Deep()
acc = model_train(model, X_train, y_train, X_test, y_test)
print(f"Final model accuracy: {acc*100:.2f}%")
```

Listing 11.11: Retrain the model of choice after cross-validation

You can reuse the `model_train()` function as it is doing all the required training and validation. This is because the training procedure doesn't change for the final model or during k-fold cross-validation.

This model is what you can use in production. Usually it is unlike training, prediction is one data sample at a time in production. The following is how we demonstrate using the model for inference by running five samples from the test set:

```
model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        y_pred = model(X_test[i:i+1])
        print(f"{X_test[i].numpy()} -> {y_pred[0].numpy()} " +
              f"(expected {y_test[i].numpy()})")
```

Listing 11.12: Evaluation of a model

Its output should look like the following:

```
[0.0265 0.044  0.0137 0.0084 0.0305 0.0438 0.0341 0.078  0.0844 0.0779
 0.0327 0.206  0.1908 0.1065 0.1457 0.2232 0.207  0.1105 0.1078 0.1165
 0.2224 0.0689 0.206  0.2384 0.0904 0.2278 0.5872 0.8457 0.8467 0.7679
 0.8055 0.626  0.6545 0.8747 0.9885 0.9348 0.696  0.5733 0.5872 0.6663
 0.5651 0.5247 0.3684 0.1997 0.1512 0.0508 0.0931 0.0982 0.0524 0.0188
 0.01   0.0038 0.0187 0.0156 0.0068 0.0097 0.0073 0.0081 0.0086 0.0095] -> [0.9583146] (expected [1.])
...

[0.034  0.0625 0.0381 0.0257 0.0441 0.1027 0.1287 0.185  0.2647 0.4117
 0.5245 0.5341 0.5554 0.3915 0.295  0.3075 0.3021 0.2719 0.5443 0.7932
 0.8751 0.8667 0.7107 0.6911 0.7287 0.8792 1.     0.9816 0.8984 0.6048
 0.4934 0.5371 0.4586 0.2908 0.0774 0.2249 0.1602 0.3958 0.6117 0.5196
 0.2321 0.437  0.3797 0.4322 0.4892 0.1901 0.094  0.1364 0.0906 0.0144
 0.0329 0.0141 0.0019 0.0067 0.0099 0.0042 0.0057 0.0051 0.0033 0.0058] -> [0.01937182] (expected [0.])
```

Output 11.4: Result of evaluation

You run the code under `torch.no_grad()` context because you sure there's no need to run the optimizer on the result. Hence you want to relieve the tensors involved from remembering how the values are computed.

The output of a binary classification neural network is between 0 and 1 (because of the sigmoid function at the end). From `encoder.classes_`, you can see that 0 means "M" and 1 means "R". For a value between 0 and 1, you can simply round it to the nearest integer and interpret the 0-1 result, i.e.,

```
y_pred = model(X_test[i:i+1])
y_pred = y_pred.round() # 0 or 1
```

Listing 11.13: Prediction using the trained model

or use any other threshold to quantize the value into 0 or 1, i.e.,

```
threshold = 0.68
y_pred = model(X_test[i:i+1])
y_pred = (y_pred > threshold).float() # 0.0 or 1.0
```

Listing 11.14: Prediction with a threshold

Indeed, round to the nearest integer is equivalent to using 0.5 as the threshold. A good model should be robust to the choice of threshold. It is when the model output exactly 0 or 1. Otherwise, you would prefer a model that seldom returns values in the middle but often returns values close to 0 or close to 1. To see if your model is good, you can use *receiver*

operating characteristic curve (ROC), which is to plot the true positive rate against the false positive rate of the model under various threshold. You can make use of scikit-learn and matplotlib to plot the ROC:

```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

with torch.no_grad():
    # Plot the ROC curve
    y_pred = model(X_test)
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    plt.plot(fpr, tpr) # ROC curve = TPR vs FPR
    plt.title("Receiver Operating Characteristics")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.show()
```

Listing 11.15: Plotting the ROC curve of a binary classification model

You may see the following. The curve is always starting from the lower left corner and ends at upper right corner. The closer the curve to the upper left corner, the better your model is.

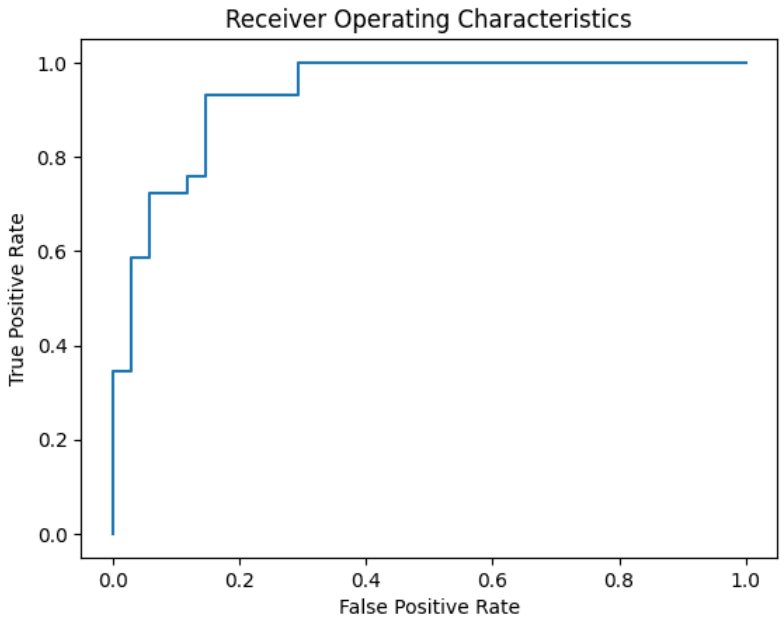


Figure 11.1: The ROC curve from the binary classification model

11.6 Complete Code

Putting everything together, the following is the complete code of the above:

```
import copy

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.metrics import roc_curve
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import LabelEncoder

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define two models
class Wide(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(60, 180)
        self.relu = nn.ReLU()
        self.output = nn.Linear(180, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.hidden(x))
        x = self.sigmoid(self.output(x))
        return x

class Deep(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
```

```
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.sigmoid(self.output(x))
        return x

# Compare model sizes
model1 = Wide()
model2 = Deep()
print(sum([x.reshape(-1).shape[0] for x in model1.parameters()]))  # 11161
print(sum([x.reshape(-1).shape[0] for x in model2.parameters()]))  # 11041

# Helper function to train one model
def model_train(model, X_train, y_train, X_val, y_val):
    # loss function and optimizer
    loss_fn = nn.BCELoss()  # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 300   # number of epochs to run
    batch_size = 10  # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_acc = - np.inf   # init to negative infinity
    best_weights = None

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
        # evaluate accuracy at end of each epoch
        model.eval()
        y_pred = model(X_val)
        acc = (y_pred.round() == y_val).float().mean()
        acc = float(acc)
        if acc > best_acc:
```

```
            best_acc = acc
            best_weights = copy.deepcopy(model.state_dict())
    # restore model and return best accuracy
    model.load_state_dict(best_weights)
    return best_acc

# train-test split: Hold out the test set for final model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores_wide = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores_wide.append(acc)
cv_scores_deep = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Deep()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (deep): %.2f" % acc)
    cv_scores_deep.append(acc)

# evaluate the model
wide_acc = np.mean(cv_scores_wide)
wide_std = np.std(cv_scores_wide)
deep_acc = np.mean(cv_scores_deep)
deep_std = np.std(cv_scores_deep)
print("Wide: %.2f%% (+/- %.2f%%)" % (wide_acc*100, wide_std*100))
print("Deep: %.2f%% (+/- %.2f%%)" % (deep_acc*100, deep_std*100))

# rebuild model with full set of training data
if wide_acc > deep_acc:
    print("Retrain a wide model")
    model = Wide()
else:
    print("Retrain a deep model")
    model = Deep()
acc = model_train(model, X_train, y_train, X_test, y_test)
print(f"Final model accuracy: {acc*100:.2f}%")

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        y_pred = model(X_test[i:i+1])
        print(f"{X_test[i].numpy()} -> {y_pred[0].numpy()} " +
              f"(expected {y_test[i].numpy()})")

    # Plot the ROC curve
    y_pred = model(X_test)
```

```
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    plt.plot(fpr, tpr) # ROC curve = TPR vs FPR
    plt.title("Receiver Operating Characteristics")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.show()
```

Listing 11.16: Complete code for the binary classification problem

11.7 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Connectionist Bench (Snoar, Mines vs. Rocks) Data Set. UCI Machine Learning Repository.
https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks)

BCELoss. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html

Label encoder. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html

ROC curve. scikit-learn documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html

Multiclass Receiver Operating Characteristic (ROC). scikit-learn examples.
https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html

11.8 Summary

In this chapter, you discovered the use of PyTorch to build a binary classification model.

You learned how you can work through a binary classification problem step-by-step with PyTorch, specifically:

- ▷ How to load and prepare data for use in PyTorch
- ▷ How to create neural network models and use k-fold cross-validation to compare them
- ▷ How to train a binary classification model and obtain the receiver operating characteristics curve for it

In the next chapter, you will do a regression project.